

CNN FOR CIFAR10 DATASET (IMAGE CLASSIFICATION)

```
In [1]: import torch
import torch.nn as nn
import torch.optim as optim

import torchvision#torchvision is an official library of PyTorch used for Comput
from torchvision.datasets import CIFAR10
```

DATASETS AND DATALOADER

```
In [2]: # Image
# ↓
# Convert image → Tensor
# ↓
# Scale pixels (0-255 → 0-1)
# ↓
# Normalize values (-1 to +1)
# ↓
# Ready for Neural
# -----
#           X-mean
# Xnew= -----
#           std
# X = pixel value
# mean = average value
# std = standard deviation
# (mean_R , mean_G , mean_B)
# (std_R , std_G , std_B)
# mean = (0.5,0.5,0.5)
# std = (0.5,0.5,0.5)
# Why 3 numbers?
# Because RGB image has 3 channels:
# Red
# Green
# Blue
# So normalization is done for each channel.
# -----
# Compose() means:
# Combine multiple transformations into one pipeline.
```

```
In [3]: from torch.utils.data import DataLoader
import torchvision.transforms as transforms

transform=transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,0.5,0.5),(0.5,0.5,0.5))
])

trainset=CIFAR10(root="./data",train=True,download=True,transform=transform)
testset=CIFAR10(root="./data",train=False,download=True,transform=transform)
```

Files already downloaded and verified
Files already downloaded and verified

```
In [4]: trainloader=DataLoader(trainset,batch_size=64,shuffle=True)
        testloader=DataLoader(testset,batch_size=64)
```

Build the CNN

```
In [5]: # # First Convolution Block
        # # nn.Conv2d(3,32,kernel_size=3,padding=1)
        # # Meaning
        # # Parameter    Meaning
        # # 3      Input channels
        # # 32     Number of filters
        # # kernel_size=3      Filter size 3x3
        # # padding=1    Adds 1 pixel border

        # Why input channel = 3 ?
        # Because RGB images have 3 channels:
        # Red
        # Green
        # Blue
        # So input shape is like:
        # 3 x H x W
        # Example:
        # 3 x 32 x 32
        # Output after convolution
        # If input =3 x 32 x 32
        # After:
        # Conv2d(3,32,3,padding=1)
        # Output becomes:
        # 32 x 32 x 32
        # Explanation:
        # 32 filters are applied
        # Each filter creates 1 feature map
        # So we get 32 feature maps

        # nn.MaxPool2d(2,2)
        # Pooling reduces image size.
        # Parameters:
        # kernel_size = 2
        # stride = 2
        # Meaning:
        # Take 2x2 region → keep max value
        # Example:
        # Input
        # 4 2
        # 6 8
        # Output = 8
        # Size reduction
        # Before pooling:
        # 32 x 32 x 32
        # After pooling:
        # 32 x 16 x 16
        # Because pooling halves width & height.

        #Second Convolution Block
        # nn.Conv2d(32,64,kernel_size=3,padding=1)
```

```

# Meaning:
# Parameter      Meaning
# 32    input feature maps
# 64    filters
# kernel_size=3 filter size
# Output:
# 64 x 16 x 16
# Then pooling:
# 64 x 8 x 8

# Third Convolution Block
# nn.Conv2d(64,128,kernel_size=3,padding=1)
# Output:
# 128 x 8 x 8
# After pooling:
# 128 x 4 x 4

# Final Feature Map
# After all layers:
# 128 x 4 x 4
# This will be flattened later before fully connected layer.

```

```

In [6]: class CNN(nn.Module):
        def __init__(self):
            super(CNN,self).__init__()

            self.conv_layers=nn.Sequential(
                # 1st Layer(convo+ReLU+Pool)
                nn.Conv2d(3,32,kernel_size=3,padding=1),#channel=3 and #filter=32
                nn.ReLU(),
                nn.MaxPool2d(2,2),#kernel_size=2, stride=2
                # 2nd Layer(convo+ReLU+Pool)
                nn.Conv2d(32,64,kernel_size=3,padding=1),
                nn.ReLU(),
                nn.MaxPool2d(2,2),#kernel_size=2, stride=2
                # 3rd Layer(convo+ReLU+Pool)
                nn.Conv2d(64,128,kernel_size=3,padding=1),
                nn.ReLU(),
                nn.MaxPool2d(2,2)#kernel_size=2, stride=2
            )

            self.fc_layers=nn.Sequential(
                nn.Linear(4*4*128,256),
                nn.ReLU(),

                nn.Linear(256,10)#10 neuron at output
            )

        def forward(self,x):
            x=self.conv_layers(x)
            x=x.view(x.size(0),-1)#Flattening
            x=self.fc_layers(x)

            return x

```

Object created

```
In [7]: model=CNN()
```

Optim and loss

```
In [8]: criterion=nn.CrossEntropyLoss()#include softmax inbuilt
optimizer=optim.Adam(model.parameters())
```

Training the CNN

```
In [9]: epochs=10

for epoch in range(epochs):
    epoch_training_loss=0.0

    for images,labels in trainloader:
        optimizer.zero_grad()

        output=model.forward(images)#Forward propagation
        loss=criterion(output,labels)
        loss.backward()#back prop....compute gradients
        optimizer.step()#Update parameters

    epoch_training_loss +=loss.item()
    print(f"epoch={epoch+1}/{epochs} & loss{epoch_training_loss/len(trainloader)}
```

```
epoch=1/10 & loss1.3605614248901376
epoch=2/10 & loss0.9273137454791447
epoch=3/10 & loss0.7407869975204053
epoch=4/10 & loss0.6109163598788668
epoch=5/10 & loss0.5066367304500412
epoch=6/10 & loss0.4093502647126727
epoch=7/10 & loss0.32255971155431873
epoch=8/10 & loss0.25607930598280315
epoch=9/10 & loss0.19721950307640884
epoch=10/10 & loss0.15881297417232754
```

Evaluation Our CNN

```
In [10]: correct_labels=0
total_labels=0
model.eval()

with torch.no_grad():
    for images , labels in testloader:
        outputs=model.forward(images)
        _,predicted=torch.max(outputs,1)

        correct_labels+=(predicted==labels).sum().item()
        total_labels+=labels.size(0)
    print(f"accuracy={correct_labels/total_labels*100}")
```

```
accuracy=75.56
```